

REST 練習 2：一個 Service、多個資源——URI 路由

Lecture

rs01 示範的是「整個 Service 只有一個 Resource」的最簡單情況（Application 直接回傳一個 Resource 實例）。但實際的 API 通常有很多種資源（航班、訂位、乘客.....），每一種都需要自己的 URL 路徑跟自己的處理邏輯——這時候 Application 就不能只回傳單一 Resource，要有一個「路由器」依照 URL 路徑決定要交給哪一個 Resource。

`CL_REST_ROUTER` 就是這個角色：`ATTACH` 方法登記「URL 樣板 → Handler Class」的對應關係，一個 Router 可以掛很多條樣板。框架收到 request 後，Router 會依樣板比對 URL 路徑，找到對應的 Resource Class 並把 request 轉交給它。

為什麼要拆成 **Application**（總機）跟 **Resource**（工人）兩層，而不是一個 **Class** 全包：這是單一職責原則（Single Responsibility）的具體實踐——Application 只管「這個 request 該交給誰」，完全不碰業務邏輯；Resource 只管「收到 request 之後怎麼處理這一種資源」，完全不管路由。這樣的分工讓每個 Resource Class 可以獨立開發、獨立測試，也方便之後新增資源時只要多寫一個 Resource Class + 多一行 `ATTACH`，不用碰其他資源的程式碼。

這題也是本課程第一次出現「一個 Application 掛兩個 Resource」的寫法，之後 rs03~rs09 的 Application Class 幾乎都是這個模式（差別只在掛幾條路徑、掛哪些 Resource）。

學習目標

- 理解為什麼「一個 SICF Service 只能掛一個 Handler Class」，但實務上一個 Service 常常要處理很多種資源（`/hello`、`/carriers`、`/flights`.....）
- 會用 `CL_REST_ROUTER : ATTACH( iv_template = 'URI樣板' iv_handler_class = '類別名稱字串' )` 把 URI Pattern 對應到 Resource Class
- 理解 router 是「收到 request 才動態 `CREATE OBJECT`」，`ATTACH` 只是註冊對照表，不會馬上建立 Resource 物件
- 能分辨「Application Class 決定路由規則」和「Resource Class 負責實際邏輯」這兩層職責不要混在一起

為什麼需要 Router

rs01 的 `ZCL_RS01_APP` 直接 `ro_root_handler = NEW zcl_rs01_hello( )`——這只在「整個 service 只有一種資源」時夠用。實務上一個 REST API 通常要同時提供好幾種資源（例如訂單、客戶、航班.....），每種資源各自有一個 Resource Class，但 SICF 一個 Service 只能指定一個 Handler Class（Application Class）。解法：Application Class 不直接處理，而是回傳一個 `CL_REST_ROUTER` 實例，router 依照 URL 路徑決定要交給哪個 Resource Class。

事前準備

ADT 端物件已由課程準備好（`$TMP`）：

- `ZCL_RS02_APP`——Application Class，改用 router
- `ZCL_RS02_HELLO`——沿用 rs01 的問候邏輯
- `ZCL_RS02_CARRIERS`——查 `SCARR` 回傳純文字清單（JSON 序列化留到 rs03）

題目需求 (對照已建好的答案物件)

1. ZCL_RS02_APP 的 GET_ROOT_HANDLER :

```
``abap DATA(lo_router) = NEW cl_rest_router( ). lo_router->attach( iv_template = '/hello'
iv_handler_class = 'ZCL_RS02_HELLO' ). lo_router->attach( iv_template = '/carriers'
iv_handler_class = 'ZCL_RS02_CARRIERS' ). ro_root_handler = lo_router. ` 注意
iv_handler_class 是字串，不是物件參考——router 內部用 CREATE OBJECT ... TYPE (class_name) 動態建立，
兩個 Resource Class 之間、甚至跟 router 之間都不需要互相 import`
```

1. ZCL_RS02_HELLO：跟 rs01 的 ZCL_RS01_HELLO 邏輯相同 (只覆寫 GET，回傳純文字問候)
2. ZCL_RS02_CARRIERS：覆寫 GET，SELECT carrid, carrname FROM scarr ORDER BY carrid INTO TABLE @DATA(lt_carrier)，用 REDUCE string() 組成每行一筆的純文字清單回傳

SICF 掛載步驟 (比照 rs01，只是換掉數值)

沿用 rs01 已建好的 /sap/bc/zrest_training 分類節點，這題只要在其底下再加一個子節點：

1. SICF 交易碼，在 zrest_training node 上右鍵 → **New Sub-Element**，Service Name 填 rs02
2. Handler List 掛 ZCL_RS02_APP (不是 ZCL_RS01_APP)
3. Activate Service
4. 測試兩個路徑：
 - http://<主機>:<port>/sap/bc/zrest_training/rs02/hello?sap-client=130
 - http://<主機>:<port>/sap/bc/zrest_training/rs02/carriers?sap-client=130

預期輸出 (範例)

/hello：

```
Hello REST! 現在伺服器時間是 14:40:12
```

/carriers：

```
AA  American Airlines
LH  Lufthansa
UA  United Airlines
...
```

團隊實務備註

- ATTACH 的 URI 樣板語法支援 {變數} (如 /carriers/{carrid}) 擷取路徑參數，但這題先只用固定路徑，路徑參數留到 rs04
- Router 用 CREATE OBJECT TYPE (字串) 動態建立物件，代表：Resource Class 名稱一旦打錯字，編譯期不會抓到，要實際呼叫那個 URL 才會發現 (通常是 404 或 dump) ——這是動態建立物件的通病，op08 學過的多型轉型是靜態繫結，這裡是完全不同的動態機制
- 一個 Application Class 可以掛任意多個 ATTACH，實務上大型 API 會用同一個 router 管理十幾個資源

思考題

1. 如果 `/carriers` 和 `/carrier` 兩個路徑都 `ATTACH` 了不同的 Resource Class，`CL_REST_ROUTER` 怎麼判斷用哪一個？（提示：看 `FIND_MATCH` / `GET_FIRST_MATCH` 的排序邏輯——精確字面比對的優先權高於帶變數的樣板）
2. `ZCL_RS02_CARRIERS` 現在回傳純文字、用 `\t` 分欄——如果呼叫端是一支前端 JS 程式，這種格式好解析嗎？（這正是 `rs03` 要解決的問題）
3. 承 `op05`「全域類別」：`ZCL_RS02_HELLO`、`ZCL_RS02_CARRIERS` 這兩個 Resource Class 彼此完全不知道對方存在，也不需要共同的父類別或介面——`router` 是怎麼做到「不用介面也能一致呼叫」的？

答案

見 `zcl_rs02_app.clas.abap`、`zcl_rs02_hello.clas.abap`、`zcl_rs02_carriers.clas.abap`（SAP 端物件 `ZCL_RS02_APP` / `ZCL_RS02_HELLO` / `ZCL_RS02_CARRIERS`）。SICF Service 路徑 `/sap/bc/zrest_training/rs02`。